

Practical-3

Aim:

To Implementation of max-heap sort algorithm

Algorithm:

CLASSMATE
 Date _____
 Page _____

Algorithm: max-heap Sort.

Input: An array A containing n elements

Output: The array sorted in ascending order using Max-heap Sort

Step 1: Start

Step 2: Start with the last non-leaf node:
 $i = n/2 - 1$

2. for each node from i down to 0, call the heapify()
3. In heapify(), compare the parent node with its left and right child.

Step 3: 1. find the largest element among the parent, left child, right child
 2. if the largest element is not Parent, swap the Parent with the largest child.
 3. Repeat the heapify process for the affected subtree

Step 4: 1. Set $i = n - 1$
 2. Swap the root element $A[0]$ with $A[i]$

Step 5: Reduce the heap size by 1

Step 6: Apply heapify() to the root node to restore the Max heap Property

Step 7: decrease i by 1

Step 8: Repeat Step 4-7 until $i = 0$

Step 9: Stop

Code:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort
    for (int i = n - 1; i > 0; i--)
    {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}
```

```

    }
}

int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = 6;

    cout << "Original array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    heapSort(arr, n);

    cout << "\nSorted array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Output:

The screenshot shows a Windows 10 desktop with a Visual Studio Code editor window. The editor is open to a C++ file named 'practical-3.cpp' in the 'dda-practicals' project. The code implements a bubble sort algorithm. The Explorer sidebar on the left shows the project structure. The Output window at the bottom displays the execution results of the program.

Visual Studio Code Explorer:

- dda-practicals
 - binarysearch.cpp
 - binarysearch.exe
 - linearsearch.cpp
 - linearsearch.exe
 - practical-3.cpp
 - practical-3.exe
 - practical1.1.cpp
 - practical1.1.exe
 - practical1.cpp
 - practical1.exe
 - practical2.cpp
 - practical2.exe

practical-3.cpp Code:

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    heapSort(arr, n);

    cout << "\nSorted array: ";

    for (int i = 0; i < n; i++)
```

Output Window:

```
PS C:\Desktop\dda-practicals> "enter program name, for example C:\Desktop\dda-practicals/a.exe"
enter program name, for example C:\Desktop\dda-practicals/a.exe
PS C:\Desktop\dda-practicals> C:\Desktop\dda-practicals/practical-3.exe
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13
PS C:\Desktop\dda-practicals>
```

Time complexity:

classmate
Date _____
Page _____

At each recursive call, we move down one level in the heap

A binary heap has approximately half the elements in the subtree we continue into

So, the recurrence is:

$$T(n) = T(n/2) + O(1)$$

By applying master's Theorem:

$$T(n) = aT(n/b) + f(n)$$

Compare:

$$T(n) = T(n/2) + 1$$

$$\therefore a=1, b=2, K=0, P=0$$

$$f(n) = 1 = O(1)$$

By calculating:

$$= n^{\log_b a}$$

$$= n^{\log_2 1}$$

$$\because \log_2 1 = 0$$

$$n^0 = 1$$

$$\text{So, } f(n) = O(n^{\log_2 1})$$

So, By master Theorem Case 2

$$T(n) = O(\log n)$$

$$\boxed{\text{Heapify} = O(\log n)}$$

So, Heap sort has n heapify operations.

$$T(n) = n \times O(\log n)$$

$$T(n) = O(n \log n)$$

For, Max-Heap sort:

$$\text{Best} = \text{Average} = \text{Worst} = O(n \log n)$$

Worst case:

$$T_{\text{worst}}(n) = T(n/2) + O(1)$$

$$a=1, b=2, f(n)=1$$

$$n^{\log_2 1} = 1$$

$$\therefore T_{\text{worst}}(n) = O(n \log n)$$

Average case:

$$T_{\text{avg}}(n) = T(n/2) + O(1)$$

$$a=1, b=2, f(n)=1$$

$$n^{\log_2 1} = 1$$

$$\therefore T_{\text{avg}}(n) = O(n \log n)$$

Heapify method

algo:

MaxHeapify(arr, size, i) {

 int largest = i; $\rightarrow 1$

 int l = (2*i); $\rightarrow 1$

 int r = (2*i+1); $\rightarrow 1$

 while (l < size && arr[l] > arr[largest]) {

 largest = l;

 }

 while (largest != i) {

 swap(arr[largest], arr[i]);

 largest = largest; $\rightarrow \log n$

 } if (largest != i) {

 swap = (arr[largest], arr[i]);

 MaxHeapify(arr, size, largest);

 }

}

HeapSort (l = size; l >= 1; l--) {

 MaxHeapify(arr, size, l);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

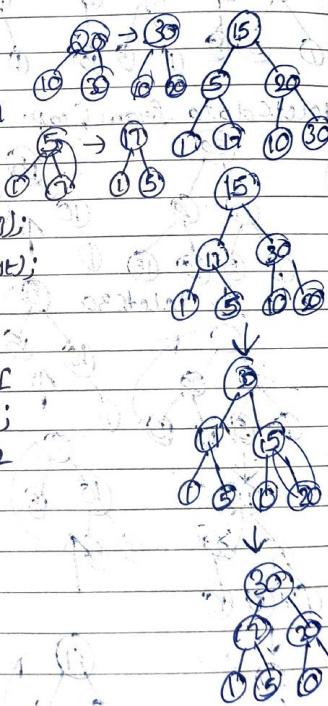
 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);

 MaxHeapify(arr, size, 1);

 swap(arr[l], arr[1]);



Time Complexity

$O(n) \times O(\log n)$

$O(n \log n)$

Result:

The Max-Heap Sort algorithm was successfully implemented and executed. The given elements were sorted in ascending order, and the time complexity was analyzed for the best, average, and worst cases. Thus, the implementation of the Max-Heap Sort algorithm was successfully done.